



Figure 3: RadioButtons



Figure 4: MessageBox



Figure 5: ComboBox and ListWidget

```
case QMessageBox::Yes:
    ui->label_2->setText("You have chosen " + item);
    break;
case QMessageBox::No:
    ui->label_2->setText("You haven't made any choice.");
    break;
}
```

A message box with 'severity information' is created. The arguments for the `QMessageBox` constructor are the parent's address, title, text to be displayed, and then a list of OR-ed `enum` members like `Yes`, `No`, `Save`, etc, which will provide the buttons on the message-box. When a message-box is executed, and the user clicks one of the buttons, the message box will disappear, and the `enum` value associated with the button will be returned—which you can use to decide what to

do, using a `switch` statement, as shown above.

Radio buttons

Radio buttons are used to select a single choice from a group—only one in a group can be selected at a time. When a radio button is selected, it emits the `clicked()` signal. Here is sample code for a slot to handle the signal:

```
void Dialog::on_radioButton_clicked(bool checked)
{
    if(checked)
        item = ui->radioButton->text();
}
```

Based on the `bool` value, you can respond accordingly. Radio buttons have a 'built-in' label; you don't need to use a separate label widget for the text.

Sometimes, we need to allow multiple choices in a group—like, the user chooses three colours from five. Here, radio buttons won't work; so use the `checkBox` widget, which gives `check/uncheck` options to enable/disable an option or features. Once again, the `clicked()` signal is emitted when the widget is clicked, with the `bool` value passed to the slot. Example code can be downloaded from the LFY site. Figures 2 to 4 are screen-shots of the application.

Besides these buttons, basic GUI applications need list widgets. We will cover two of them—combo box and list widget.

The combo box provides a drop-down list, and shows only the current selected item—which is an advantage, since it uses minimum space on the window. In Qt, this widget is provided by the `QComboBox` class. Items can be inserted and deleted at run-time too. A signal is generated when the item selection is changed. If you need the user to see the full list, then use `QListWidget`. Look at Figure 5. The combo box shows one item, `QListWidget` shows the full list. The following code inserts items in them:

For combo box:

```
ui->comboBox->addItem(ui->lineEdit->text());
```

For list:

```
QListWidgetItem * item = new QListWidgetItem( ui->lineEdit_2-
>text(), ui->listWidget);
ui->listWidget->insertItem(ui->listWidget->count(), item);
```



By: Manoj Kumar

The author is a freelance developer and trainer. He leads a team in Linux kernel programming, Linux administration, cluster computing, embedded systems and QT/GTK programming on Linux. You can reach him at manoj@nixitsolutions.com and read his blog, www.open4india.blogspot.com, to share your Linux knowledge.